

Course Project Phase 1

SQL Server Database Development

Team members : Sahasri Nunemuntala(N10010782), Kelvin Idoko (N01777723) and Hassana Abdullahi(N10000326)

Business Case: Music Streaming Management System

1. Introduction

This project involves the design and implementation of a fully functional SQL Server-based database system for a **Music Streaming Management Platform** using Microsoft SQL Server 2016 and Transact-SQL (T-SQL) only.

The system simulates a real-world digital music platform where users can stream songs, create playlists, follow artists, and manage subscriptions. The database will be fully normalized, secure, and optimized with no GUI, no ORMs, and no frontend application.

This project demonstrates full understanding of relational database systems including schema design, procedural logic, transactions, security, indexing, and system optimization.

2. Project Objectives

Upon completion of the project, the system will demonstrate the ability to:

- Design a fully normalized relational database (3NF or higher)
 - Implement stored procedures, triggers, functions, views, and dynamic SQL
 - Manage transaction control with error handling and concurrency control
 - Apply role-based security using GRANT, REVOKE, and DENY
 - Optimize queries using indexing and execution plan analysis
 - Demonstrate a fully integrated backend system using T-SQL only
-

3. Business Case Requirements

3.1 Problem Domain

The system manages a music streaming platform where users can:

- Stream songs
- Create and manage playlists
- Follow artists and explore albums
- Track listening history
- Manage subscriptions (Free / Premium)

Artists can upload songs and albums, while administrators manage platform content and moderation.

3.2 Required Entities

The system includes the following interrelated entities:

- Users
 - Roles
 - Songs
 - Artists
 - Albums
 - Genres
 - Playlists
 - PlaylistSongs (junction table)
 - ListeningHistory
 - Subscriptions
-

3.3 User Roles

The system defines the following roles:

- Admin – full system control and security management
 - User – stream music, create playlists
 - Artist – upload and manage songs/albums
 - Moderator – manage content and enforce platform rules
-

3.4 User Operations Per Role

Admin:

- Manage users and roles
- Approve or remove content
- Manage system security

User:

- Stream songs
- Create and manage playlists
- View listening history

Artist:

- Upload songs and albums
- Update metadata of their content

Moderator:

- Remove inappropriate content
 - Monitor activity logs
-

3.5 Business Workflows

Workflow 1: Song Streaming Tracking

- User plays a song
 - System logs entry into ListeningHistory
 - Updates usage statistics
-

Workflow 2: Playlist Management

- User creates playlist
 - Songs are added with validation
 - Duplicate prevention enforced
-

Workflow 3: Subscription Management

- User upgrades subscription

- Transaction updates user status and subscription records
 - Ensures data consistency using COMMIT/ROLLBACK
-

3.6 Business Rules

- A user cannot add duplicate songs to a playlist
 - A song must belong to exactly one artist
 - Only active users can stream songs
 - Subscription must be valid to access premium features
 - Songs cannot be deleted if referenced in playlists or history
 - All streaming activity must be logged
-

4. Technical Environment

- SQL Server 2016 (vApp)
 - SQL Server Management Studio (SSMS)
 - GitHub (private repository)
 - T-SQL only implementation
 - No GUI or external frameworks allowed
-

5. Functional Requirements

All database components will be implemented using T-SQL scripts and organized into structured folders.

5.1 Schema Design (Minimum 6 tables)

- Fully normalized (3NF or higher)
 - Primary keys and foreign keys defined
 - Data types, constraints, and defaults included
 - Proper relationship mapping
-

5.2 Procedural Logic Requirements

Object Type	Quantity	Notes
Stored Procedures	≥ 5	Includes 1 dynamic SQL procedure
User Defined Functions	≥ 2	Scalar or table-valued
Views	≥ 3	Role-based or abstraction views
Triggers	≥ 3	AFTER / INSTEAD OF triggers
Cursors	≥ 2	One must be dynamic cursor

All procedures will include:

- Parameterization
 - Validation logic
 - Flow control (IF, WHILE, etc.)
-

5.3 Security & Access Control

- Minimum 3 roles (Admin, User, Artist, Moderator)
 - GRANT, REVOKE, DENY implementation
 - Row-level security using views
 - Role-based access testing scenarios included
-

5.4 Transaction Management

All INSERT/UPDATE/DELETE operations include:

- BEGIN TRANSACTION
 - COMMIT / ROLLBACK
 - TRY...CATCH error handling
 - Concurrency control simulation
-

5.5 Indexing & Performance

Minimum indexes required:

- 1 Clustered Index (e.g., SongID or UserID)
- 1 Non-Clustered Index (e.g., ArtistID in Songs)
- 1 Filtered or INCLUDE Index (e.g., ListeningHistory)

Performance validation:

- Execution plan comparison before/after indexing
- Query optimization analysis

ER DIAGRAM

